Nightreign Recognizer v11: Initial Screenshot-to-Map Crop

A compact explanation of how the first crop works in the current recognizer: from full screenshot, to map circle, to square panel crop, and only then to the smaller recognition square.

Key idea. The recognizer performs two different crops. First it extracts the full square map panel from the screenshot. After that, it crops a smaller centered square from inside that panel for matching.

1. Full pipeline shape

```
Full screenshot (selected monitor)
        |
        v
Detect map circle
        |
        +-- if blue ring is trusted:
        |       crop_box = square from circle
        |       skip edge refinement
        |
        +-- otherwise:
                crop_box = refined square from circle
        |
        v
cropped_panel = full_img[y0:y1, x0:x1]   <- first crop
        |
        v
focus_square = centered square inside cropped_panel
        |
        v
recognition features + ranking
```

The first crop is the full square map panel. The second crop is the smaller centered recognition square.

2. Stable fallback circle

The script always keeps a stable fallback estimate based on the screenshot size. If detection is weak or drifts too far, it falls back to this anchor.

```
fallback.cx = w * 0.76
fallback.cy = h * 0.50
fallback.r  = min(h, w) * 0.32
```

So the fallback circle is centered at 76% of the screenshot width and 50% of the screenshot height, with radius equal to 32% of the smaller screen dimension.

3. Search region used for circle detection

With the default settings, detection searches only the right half of the screenshot, then downsizes that region by a factor of 0.5 before looking for circles.

```
x0 = w // 2
y0 = 0
x1 = w
y1 = h
roi = full_img[y0:y1, x0:x1]

scale = 0.5
small_w = round((x1 - x0) * scale)
small_h = round((y1 - y0) * scale)
```

4. Blue-ring circle detection

The current v11 logic first tries to detect the blue outer ring of the map. It thresholds HSV values in a blue band, then runs Hough circle detection on that mask.

```
lower = [88, 20, 70]
upper = [132, 255, 255]
```

Each blue candidate found in the resized ROI is mapped back to full-screen coordinates with these equations:

```
full_cx = x0 + (cx / scale)
full_cy = y0 + (cy / scale)
full_r  = r / scale
```

The blue candidate score prefers circles near the fallback center and strongly rewards larger radii:

```
score =
    abs(full_cx - fallback.cx) / w
  + abs(full_cy - fallback.cy) / h
  - 0.40 * (full_r / min(h, w))
```

The best blue circle is trusted only if it stays close to the fallback anchor:

```
abs(center_dx) <= w * 0.03
abs(center_dy) <= h * 0.08
0.92 <= radius_ratio <= 1.10
```

5. Grayscale Hough fallback

If blue-ring detection fails or drifts too much, the recognizer runs a more generic grayscale Hough-circle pass.

```
full_cx = x0 + (cx / scale)
full_cy = y0 + (cy / scale)
full_r  = r / scale

size_bonus = 0.30 * (full_r / min(h, w))

score =
    abs(full_cx - fallback.cx) / w
  + abs(full_cy - fallback.cy) / h
  - size_bonus
```

Even then, the script is conservative. If the grayscale circle drifts too far from the fallback, it is discarded. If it is acceptable, the script only blends 25% of the detected shift and radius change into the fallback estimate.

```
blended_cx = fallback.cx + center_dx * 0.25
blended_cy = fallback.cy + center_dy * 0.25
blended_r  = fallback.r  + (chosen.r - fallback.r) * 0.25
```

6. Turn the circle into a square crop box

Once the final circle is chosen, the script turns it into a square using panel_scale = 1.12.

```
side = 2.0 * r * 1.12
half = side / 2.0

box.x0 = round(cx - half)
box.y0 = round(cy - half)
box.x1 = round(cx + half)
box.y1 = round(cy + half)
```

This creates the first estimate of the square map panel.

7. Clip the box to the screenshot bounds

```
x0 = max(0, min(w - 1, x0))
y0 = max(1, min(h - 1, y0))
x1 = max(1, min(w,     x1))
y1 = max(1, min(h,     y1))
```

8. Two possible first-crop paths

If the blue ring is trusted	The square from the circle is used directly. Edge refinement is skipped, because the blue ring is treated as the most reliable anchor.
If the blue ring is not trusted	The script refines left, right, top, and bottom edges around the estimated square using brightness and Sobel gradients, then rebuilds a square from those refined edges.

In either case, the actual first crop is just the screenshot sliced by the chosen square box:

```
cropped_panel = full_img[y0:y1, x0:x1]
```

9. Then the second crop happens

Only after the full panel crop exists does the recognizer take the smaller centered square used for matching.

```
side = round(min(h_panel, w_panel) * 0.69)
x0   = (w_panel - side) // 2
y0   = (h_panel - side) // 2

focus_square = cropped_panel[y0:y0+side, x0:x0+side]
```

10. Why tiny errors in the first crop hurt recognition

The matcher compares blurred luminance, gradient magnitude, and perceptual hash against cached references. Even a small drift in the first crop changes coastlines, roads, and landmarks enough to significantly affect ranking. That is why a crop that looks only slightly wrong can still break recognition badly.

Equation summary

```
Fallback circle:
  cx_f = 0.76 w
  cy_f = 0.50 h
  r_f  = 0.32 min(h, w)

Panel square from chosen circle:
  side = 2 r * 1.12
  x0 = round(cx - side/2)
  y0 = round(cy - side/2)
  x1 = round(cx + side/2)
  y1 = round(cy + side/2)

First crop:
  cropped_panel = full_img[y0:y1, x0:x1]

Second crop:
  focus_square = centered 0.69-side square inside cropped_panel
```

Document generated from the current v11 crop explanation requested in chat.